

Documento de definición del proyecto final

Proyecto: Página web Corazón Sabio

Fase: Planeación

Tipo de proyecto: Sitio web escolar de presentación y catálogo de productos

Producto principal: Helados veganos, sin lácteos y sin azúcar refinada

1. Definición del problema y alcance

Problema real

Corazón Sabio necesita una página web clara, atractiva y funcional para presentar su propuesta de valor: helados conscientes elaborados con ingredientes de origen vegetal, sin lácteos y sin azúcar refinada. El problema principal es que, sin una presencia digital organizada, los posibles clientes no pueden conocer fácilmente la marca, entender sus beneficios, revisar los sabores disponibles ni iniciar una intención de compra o contacto.

Además, al tratarse de un proyecto escolar, la página debe demostrar que la solución no es solo visual, sino también lógica y técnica: debe tener una estructura de navegación comprensible, componentes reutilizables, datos organizados y una ruta clara para crecer hacia funciones como carrito, blog o pedidos.

Usuarios afectados

Los usuarios principales son:

- Clientes potenciales: personas interesadas en postres más saludables, veganos o sin azúcar refinada.
- Personas con restricciones alimentarias: usuarios que evitan lácteos o buscan opciones de origen vegetal.
- Equipo del proyecto escolar: estudiantes responsables de diseñar, desarrollar y presentar la solución.
- Evaluadores o docentes: personas que revisarán si la página tiene lógica, planeación técnica y uso adecuado de herramientas computacionales.

Alcance del proyecto

La aplicación web incluirá:

- Página principal con identidad de marca, imagen destacada y mensaje central.
- Sección de filosofía de la marca.

- Catálogo de sabores disponibles con nombre, descripción, precio e imagen.
- Componentes reutilizables para encabezado, títulos de sección y tarjetas de producto.
- Navegación hacia secciones o páginas proyectadas como blog y carrito.
- Diseño responsivo para computadora y celular.
- Base de datos inicial local mediante un archivo de productos en TypeScript.
- Preparación técnica para agregar carrito de compras simulado en una fase posterior.

Quedará fuera del alcance inicial:

- Pagos reales con tarjeta o pasarela bancaria.
- Sistema de usuarios con inicio de sesión.
- Administración real de inventario.
- Panel administrativo para modificar productos desde la web.
- Envío automático de pedidos a una tienda real.
- Backend complejo o base de datos externa en la primera versión.

Esta delimitación permite construir una solución viable dentro del tiempo escolar, sin prometer funciones que requieren seguridad, infraestructura o permisos adicionales.

2. Descomposición en etapas

Etapas 1: Análisis del problema y definición del contenido

Objetivo: identificar qué debe comunicar la página y qué datos necesita mostrar.

Actividades:

- Definir la identidad de Corazón Sabio.
- Determinar el público objetivo.
- Escribir textos principales: propuesta de valor, filosofía y descripciones de productos.
- Definir la información mínima de cada producto: id, nombre, descripción, precio e imagen.

Dependencias: ninguna, es la base del proyecto.

Checkpoint de verificación:

- Existe una lista clara de productos.
- El mensaje central de la marca se entiende en una frase.

- Se confirma qué funciones sí entran y cuáles quedan fuera.

Riesgo: que el proyecto quiera abarcar demasiadas funciones.

Mitigación: mantener una primera versión enfocada en página principal, catálogo y estructura escalable.

Etapla 2: Diseño de arquitectura y navegación

Objetivo: decidir cómo se organizará la aplicación antes de programar.

Actividades:

- Definir las secciones principales: inicio, filosofía y productos.
- Definir rutas futuras: blog y carrito.
- Separar la interfaz en componentes reutilizables.
- Establecer el flujo del usuario: entrar, conocer la marca, explorar productos y seleccionar uno.

Dependencias: requiere tener definido el contenido de la etapa 1.

Checkpoint de verificación:

- La navegación principal tiene sentido.
- Cada sección cumple una función específica.
- Los componentes no repiten código innecesario.

Riesgo: que la página se vuelva confusa si se agregan secciones sin orden.

Mitigación: usar una arquitectura por componentes y mantener una jerarquía visual clara.

Etapla 3: Construcción visual de la interfaz

Objetivo: implementar la primera versión visible de la página.

Actividades:

- Crear el encabezado con logo y navegación.
- Crear el hero con imagen de marca y llamado a ver productos.
- Crear la sección de filosofía.
- Crear la cuadrícula de productos.
- Aplicar estilos con Tailwind CSS.
- Asegurar que el diseño sea responsivo.

Dependencias: requiere arquitectura definida en la etapa 2.

Checkpoint de verificación:

- La página se visualiza correctamente en escritorio y móvil.
- Las imágenes cargan sin romper el diseño.
- Los productos aparecen con datos completos.

Riesgo: que las imágenes tengan tamaños distintos y deformen las tarjetas.

Mitigación: usar dimensiones controladas, object-cover y textos con longitud razonable.

Etapla 4: Lógica de datos y preparación del carrito

Objetivo: organizar los productos como datos reutilizables y preparar la lógica para interacciones futuras.

Actividades:

- Guardar productos en una estructura de datos tipada.
- Recorrer la lista de productos para generar tarjetas automáticamente.
- Definir el comportamiento esperado del botón "Ver producto".
- Diseñar la lógica futura de carrito: agregar, actualizar cantidad, calcular subtotal y validar pedido.

Dependencias: requiere la interfaz de productos de la etapa 3.

Checkpoint de verificación:

- Al agregar un producto al archivo de datos, aparece automáticamente en la página.
- Cada producto conserva el mismo formato visual.
- La lógica del carrito queda documentada aunque no se conecte a pagos reales.

Riesgo: errores por datos incompletos, como productos sin precio o imagen.

Mitigación: usar TypeScript para definir campos obligatorios y validar datos antes de mostrarlos.

Etapla 5: Pruebas, ajustes y despliegue

Objetivo: verificar que la aplicación funciona y publicarla.

Actividades:

- Probar navegación.
- Revisar responsividad.
- Verificar que no existan errores de compilación.

- Subir el código a GitHub.
- Desplegar la página en Vercel.

Dependencias: requiere una versión funcional de las etapas anteriores.

Checkpoint de verificación:

- El comando de construcción se ejecuta correctamente.
- El sitio publicado carga sin errores.
- La URL final puede compartirse con el docente o evaluadores.

Riesgo: que la página funcione localmente pero falle al desplegarse.

Mitigación: ejecutar pruebas de build antes del despliegue y revisar rutas de imágenes en la carpeta `public`.

3. Lógica algorítmica

Flujo principal de la aplicación

El funcionamiento central de Corazón Sabio puede entenderse como un flujo de presentación, exploración y selección:

1. El usuario entra a la página.
2. El sistema carga los estilos, imágenes y componentes principales.
3. El sistema obtiene la lista de productos desde el archivo de datos.
4. La página muestra el encabezado y la sección inicial.
5. El usuario puede desplazarse hacia productos.
6. El sistema recorre la lista de productos y crea una tarjeta por cada sabor.
7. El usuario revisa nombre, descripción, precio e imagen.
8. Si el usuario selecciona un producto, el sistema puede mostrar más detalles o enviarlo al carrito en una versión futura.

Pseudocódigo

```
INICIAR aplicacion CorazonSabio

CARGAR configuracion visual
CARGAR componentes principales:
  Header
  SectionTitle
  ProductCard

CARGAR listaProductos desde archivo products
MOSTRAR Header con logo y navegacion
```

```

MOSTRAR seccion Inicio con imagen de marca y boton "Ver productos"
MOSTRAR seccion Filosofia con descripcion de la marca

SI listaProductos esta vacia ENTONCES
    MOSTRAR mensaje "No hay productos disponibles por el momento"
SINO
    PARA cada producto EN listaProductos HACER
        VALIDAR que producto tenga:
            id
            nombre
            descripcion
            precio
            imagen

        SI algun dato obligatorio falta ENTONCES
            NO mostrar tarjeta incompleta
            REGISTRAR error para correccion
        SINO
            CREAR ProductCard con los datos del producto
            MOSTRAR nombre, descripcion, precio e imagen
        FIN SI
    FIN PARA
FIN SI

CUANDO usuario presiona "Ver productos"
    MOVER vista hacia la seccion Productos
FIN CUANDO

CUANDO usuario presiona "Ver producto"
    SI existe pagina de detalle ENTONCES
        ABRIR detalle del producto seleccionado
    SINO SI existe carrito ENTONCES
        AGREGAR producto al carrito
        ACTUALIZAR total
    SINO
        MANTENER boton como accion visual de prototipo
    FIN SI
FIN CUANDO

FINALIZAR renderizado de la pagina

```

Lógica futura del carrito

```

INICIAR carrito como lista vacia

FUNCION agregarProducto(productoSeleccionado)
    BUSCAR productoSeleccionado en carrito

    SI producto ya existe EN carrito ENTONCES
        AUMENTAR cantidad en 1
    SINO
        AGREGAR producto con cantidad = 1
    FIN SI

    CALCULAR totalCarrito
FIN FUNCION

FUNCION calcularTotal(carrito)
    total = 0

    PARA cada item EN carrito HACER
        subtotal = item.precio * item.cantidad
        total = total + subtotal
    FIN PARA

    RETORNAR total
FIN FUNCION

FUNCION validarPedido(carrito, datosCliente)
    SI carrito esta vacio ENTONCES
        MOSTRAR error "Agrega al menos un producto"
        DETENER pedido

```

```

FIN SI

SI nombre, telefono o direccion estan vacios ENTONCES
    MOSTRAR error "Completa tus datos de contacto"
    DETENER pedido
FIN SI

CONFIRMAR pedido simulado
FIN FUNCION

```

Casos alternativos y casos borde

- Lista de productos vacía: la página debe mostrar un mensaje en lugar de una cuadrícula vacía.
- Imagen faltante: debe usarse una imagen de respaldo o evitar mostrar una tarjeta rota.
- Precio inválido: si el precio es menor o igual a cero, el producto no debe mostrarse como disponible.
- Pantalla pequeña: la cuadrícula debe cambiar de tres columnas a una columna.
- Usuario intenta comprar sin productos: el carrito debe bloquear el pedido.
- Datos de contacto incompletos: el sistema debe pedir corrección antes de confirmar.
- Ruta no encontrada: si blog o carrito aún no existen, se debe crear una página temporal o ajustar la navegación.

Justificación del diseño lógico

Se elige una lógica basada en datos y componentes porque permite que la página sea más mantenible. En lugar de escribir manualmente una tarjeta por cada producto, el sistema recorre una lista y genera la interfaz de forma automática. Esto reduce duplicación y errores.

La complejidad del catálogo es lineal, es decir, si hay n productos, la página crea n tarjetas. Para un catálogo pequeño escolar esto es eficiente y suficiente. Una alternativa sería usar una base de datos desde el inicio, pero eso aumentaría la complejidad técnica sin ser necesario para la primera versión. La estructura actual permite migrar a una base de datos después si el proyecto crece.

4. Selección y justificación de herramientas

Stack propuesto

Herramienta	Función dentro del proyecto	Justificación
Next.js	Framework principal de la aplicación web	Permite crear páginas con React, organizar rutas y preparar el proyecto para despliegue profesional en Vercel.
React	Construcción de componentes visuales	Facilita dividir la interfaz en piezas reutilizables como Header, ProductCard y SectionTitle.
TypeScript	Tipado de datos y prevención de errores	Ayuda a definir la estructura de un producto y reduce errores por datos incompletos o mal escritos.

Herramienta	Función dentro del proyecto	Justificación
Tailwind CSS	Estilos y diseño responsivo	Permite construir una interfaz visual consistente y adaptable sin crear demasiados archivos CSS manuales.
GitHub	Control de versiones y respaldo del código	Guarda el historial del proyecto, permite volver a versiones anteriores y facilita entregar evidencia del desarrollo.
Vercel	Hosting y despliegue web	Es compatible de forma directa con Next.js y permite publicar el sitio con una URL accesible.
Asistente de IA para programación	Apoyo en planeación, depuración y generación de código	Ayuda a acelerar el desarrollo, pero las decisiones finales deben revisarse para evitar errores o soluciones no entendidas.
Imágenes en carpeta public	Recursos visuales de marca y productos	Permiten cargar imágenes estáticas sin depender de servidores externos.

Justificación técnica frente a alternativas

Se selecciona Next.js en lugar de HTML, CSS y JavaScript simples porque el proyecto necesita crecer de una página informativa hacia una aplicación con rutas como blog y carrito. Next.js ofrece una estructura más ordenada para esa evolución.

Se selecciona React porque la página maneja elementos repetidos, especialmente las tarjetas de productos. Con componentes, una misma estructura visual puede reutilizarse para varios sabores sin duplicar código.

Se selecciona TypeScript porque el catálogo depende de datos consistentes. Si un producto no tiene nombre, precio o imagen, TypeScript ayuda a detectar el problema durante el desarrollo.

Se selecciona Tailwind CSS porque permite aplicar estilos responsivos de manera rápida y consistente. Para un proyecto escolar, esto reduce tiempo de configuración y mantiene el diseño legible.

Se selecciona Vercel frente a opciones como Netlify porque Vercel está especialmente optimizado para proyectos Next.js. También automatiza el despliegue cuando el repositorio se conecta con GitHub.

Se selecciona GitHub porque el proyecto necesita evidencia de proceso, historial de cambios y control de versiones. Esto también protege el trabajo ante errores locales.

Límites técnicos y riesgos de seguridad

- El proyecto inicial no debe almacenar datos personales sensibles si no existe backend seguro.
- Un carrito simulado no debe procesar pagos reales.
- Las variables de entorno, si se agregan después, no deben subirse al repositorio.
- Las imágenes deben optimizarse para evitar carga lenta.
- El uso de IA debe verificarse manualmente, porque puede sugerir código que compile pero no cumpla el objetivo escolar.
- Si se agregan formularios, deben validarse los campos para evitar entradas vacías o incorrectas.

5. Criterios de éxito

El proyecto se considerará exitoso si cumple con los siguientes criterios observables:

Funcionamiento

- La página carga sin errores en el navegador.
- El encabezado muestra correctamente el logo y la navegación.
- La sección principal comunica de forma clara qué es Corazón Sabio.
- La sección de productos muestra al menos tres sabores con imagen, descripción y precio.
- El botón "Ver productos" lleva al usuario a la sección correspondiente.
- El diseño se adapta correctamente a celular y computadora.

Calidad técnica

- El código está organizado en componentes reutilizables.
- Los productos están almacenados en una estructura de datos clara.
- No hay duplicación innecesaria de tarjetas de producto.
- El proyecto puede ejecutarse localmente con `npm run dev`.
- El proyecto puede construirse para producción con `npm run build`.
- El repositorio en GitHub contiene cambios organizados y descriptivos.

Experiencia de usuario

- El usuario entiende en menos de 10 segundos que la marca vende helados conscientes.
- El usuario puede encontrar los productos sin confusión.
- Los textos son breves, legibles y coherentes con la identidad de la marca.
- Las imágenes apoyan la decisión de compra y no rompen el diseño.

Métricas concretas

Criterio	Meta
Tiempo de carga inicial	Menor a 3 segundos en una conexión normal
Productos visibles	Mínimo 3 productos completos
Errores visuales graves	0 en escritorio y móvil

Criterio	Meta
Errores de compilación	0 al ejecutar build
Secciones obligatorias del sitio	Inicio, filosofía y productos
Componentes reutilizables	Mínimo 3 componentes
Claridad del flujo	El usuario puede llegar al catálogo desde el inicio con un clic

Indicadores de cumplimiento escolar

El proyecto demuestra estrategias algorítmicas porque descompone el funcionamiento en pasos: cargar datos, validar productos, recorrer la lista, renderizar tarjetas y preparar acciones futuras como carrito.

El proyecto demuestra uso de herramientas computacionales porque conecta cada tecnología con una función específica: Next.js para estructura, React para componentes, TypeScript para datos seguros, Tailwind CSS para estilos, GitHub para control de versiones y Vercel para publicación.

Conclusión

La página web Corazón Sabio es un proyecto viable porque tiene un alcance claro, una arquitectura ordenada y una lógica que puede crecer de forma progresiva. La primera versión se enfoca en presentar la marca y mostrar productos, mientras que las fases posteriores pueden añadir carrito, blog y formularios sin reconstruir toda la aplicación.

Esta planeación evita comenzar a programar a ciegas, ya que define problema, usuarios, límites, etapas, herramientas, riesgos, lógica algorítmica y criterios de éxito antes de continuar con el desarrollo.